

YOUR COMPLETE GUIDE TO BECOME
AN LLM ENGINEER

Learn LLM from Scratch to Production .NET C# EDITION

**168
DAYS**

DAY-BY-DAY
ROADMAP

Build real-world LLM applications using
C# and .NET — from fundamentals
to production deployment.



C# Fundamentals

Modern C# and .NET concepts



LLM Fundamentals

Models, prompting, embeddings,
and more



LLM Apps & Agent Systems

Build chat apps, agents and
tool use with C#



RAG & Vector Search

Work with your own data
using semantic search



Production Deployment

Docker, Azure, monitoring
and scaling

```
1 using System;  
2 using System.Net.Http;  
3 using System.Text.Json;  
4  
5 public class LlmService  
6 {  
7     public async Task<string>  
8         GenerateResponseAsync(  
9             string prompt)  
10     {  
11         // Call LLM API  
12         return await ...;  
13     }  
14 }
```

... .NET 8/9
ASP.NET Core
Entity Framework
SQL Server
Blazor
MVC

... .NET 8/9
ASP.NET Core
Entity Framework
SQL Server
Blazor
MVC

</>
C#



PHASE 1

C# Fundamentals

Day 1 – 28



PHASE 2

LLM Fundamentals

Day 29 – 56



PHASE 3

LLM Apps & Agents

Day 57 – 112



PHASE 4

RAG & Production

Day 113 – 168

WRITER

MTHREE MOLLIK

C# / .NET 10 EDITION — FREE SAMPLE

Learn LLM from Scratch to Production

The Complete Path to Becoming an LLM Engineer

A hands-on, practitioner-grade book for .NET developers — every chapter pairs theory, real C# code, and a working project, built with TorchSharp, ONNX Runtime GenAI, and the official MCP C# SDK

Author: MTHREE MOLLIK

Companion: 168-Day LLM Engineer Roadmap

GitHub: github.com/MthreeMollik/Learn-LLM-from-Scratch-to-Production

C# / .NET 10 Edition · September 2026

FREE SAMPLE

What's Inside This Preview

This is a free sample of the **.NET C# Edition** of **Learn LLM from Scratch to Production** — it includes the complete Table of Contents (all 25 chapters + 3 appendices) and the full text of Chapter 1. The complete book is a ~260-page, hands-on curriculum for .NET developers — build a GPT in pure C#, train it, fine-tune it with TorchSharp, wire it into an agent with the official MCP C# SDK, and ship it to production, with every code block written to run, not just read.

[Get the Full Book on Gumroad →](#)

Table of Contents

FRONT MATTER

Chapter 1 — Introduction: How This Journey Works (.NET Edition)

PART I — FOUNDATIONS (PURE C#)

Chapter 2 — The Minimum Math You Need: Derivative, Gradient, Chain Rule

Chapter 3 — Neural Networks & Backpropagation: micrograd in C#

Chapter 4 — The Start of Language Modeling: Bigram to MLP (makemore in C#)

Chapter 5 — Training Dynamics: Activations, Gradient Flow, Normalization

Chapter 6 — Transformers & Self-Attention: Building GPT by Hand in C#

PART II — TOKENIZATION & PRETRAINING (PURE C#)

Chapter 7 — Tokenization: BPE by Hand in C#

Chapter 8 — Pretraining a Language Model (nanoGPT in C#)

Chapter 9 — Understanding Training Deeply: Scaling Laws, Hyperparameters

Chapter 10 — Modern Architecture: RoPE, RMSNorm, GQA, KV Cache in C#

PART III — APPLICATION & AGENT DEVELOPMENT

Chapter 11 — LLM APIs & Prompt Engineering

Chapter 12 — Tool Use & the Agent Loop

Chapter 13 — RAG (Retrieval-Augmented Generation) by Hand

Chapter 14 — Building Your First Production Product

PART IV — FINE-TUNING & CUSTOMIZATION (HYBRID: TORCHSHARP)

Chapter 15 — The .NET Model Ecosystem (HuggingFace-equivalent tooling)

Chapter 16 — Fine-tuning with LoRA & QLoRA in C# (TorchSharp)

Chapter 17 — Alignment: RLHF & DPO in C#

Chapter 18 — Quantization & Local Inference

PART V — MCP, AGENTS, MEMORY

Chapter 19 — Model Context Protocol (MCP) in C#

Chapter 20 — Advanced Agent Architecture

Chapter 21 — Memory Systems: From Context Engineering to Letta-style Memory

PART VI — PRODUCTION ENGINEERING

Chapter 22 — Model Serving: vLLM Concepts & .NET Serving Patterns

Chapter 23 — Evaluation & Observability

Chapter 24 — Safety & Security

Chapter 25 — Portfolio & What's Next (.NET LLM Engineer path)

APPENDIX A

Appendix A — Glossary

APPENDIX B

Appendix B — Complete Resource List (.NET/C# Edition)

APPENDIX C

Appendix C — Environment Setup Guide (.NET 10)

Chapter 1 — Introduction: How This Journey Works (.NET Edition)

"You don't understand a thing until you can build it." — this is the one rule this book follows, and the one rule that survives the trip from Python to C# unchanged.

Why this edition exists

The original *Learn LLM from Scratch to Production* is written against Python and PyTorch — which makes sense, since that's where the LLM research world lives. But most of the world's production software isn't written in Python. A huge amount of it is written in C#, running on ASP.NET Core services, behind enterprise systems, inside Unity games, on Azure. If you're one of those developers, you've likely hit the same wall as everyone else trying to learn this field: every tutorial assumes you're fluent in `import torch`, and the moment you're not, you're stuck copy-pasting Python you don't fully trust into a codebase you do.

This edition exists to remove that wall. Every concept from the original book — derivatives, backpropagation, attention, tokenization, pretraining, fine-tuning, agents, memory, production serving — is rebuilt here in **C# on .NET Core 10**. Not translated line-by-line from Python; *re-implemented*, using the idioms, types, and tooling a .NET developer already trusts: records, `Span<T>`, operator overloading, `HttpClient`, ASP.NET Core Minimal APIs, and where real tensor computation is unavoidable, TorchSharp and ONNX Runtime — the two serious bridges between .NET and the tensor-computing world.

Who this book is for

This edition assumes you're an **experienced C# / .NET developer** — comfortable with classes, generics, LINQ, and reading a stack trace. No prior machine learning experience is required — we build that from zero, the same way the original book does. What you do need:

- Solid C# (interfaces, generics, operator overloading, `async` / `await`)
- .NET Core 10 SDK installed (see Appendix C for the exact setup)

- High-school-level algebra (variables, functions, graphs)
- A computer and an internet connection — the early chapters need no GPU at all; later chapters that use TorchSharp benefit from one but don't strictly require it
- Patience — because, exactly like the original, we don't take shortcuts

An honest scoping note, up front

Two decisions shape every chapter in this book, and it's worth being upfront about them before you invest six months in this material:

Chapters 2 through 10 — the foundations — are pure C#. No NuGet ML package, no TorchSharp, no ONNX. This isn't a limitation; it's the same choice the Python original makes. The Python book doesn't reach for PyTorch to explain backpropagation either — it hand-rolls a scalar autograd engine (`micrograd`) first, then a from-scratch GPT, precisely so nothing is hidden behind a framework call. This edition does the same thing in C#: you'll write a `Value` class with operator overloading that builds its own computation graph, and later a `Matrix` / `Tensor` type backed by plain arrays. Every gradient you compute, you'll be able to step through in the Visual Studio debugger, watching numbers change one line at a time.

Chapters 15 through 23 — the production ecosystem — are a hybrid, and I'll tell you exactly where the seams are. This is where honesty matters most. Python's LLM tooling (HuggingFace `transformers`, PEFT, TRL, bitsandbytes, vLLM) is enormous, mature, and has no full .NET equivalent. Pretending otherwise would do you a disservice. So, chapter by chapter, we do one of two things: where .NET has a genuinely usable path — **TorchSharp** for real tensor autograd and LoRA/DPO training, **ONNX Runtime GenAI** for quantized local inference, the **official MCP C# SDK** for Model Context Protocol, first-party .NET SDKs for calling Anthropic/OpenAI — we build it and run it, for real, no shortcuts. Where there is a genuine gap — bitsandbytes-style QLoRA quantization-aware training, TRL's full PPO/RLHF loop, vLLM's PagedAttention serving engine — the chapter says so plainly, explains the technique conceptually so you understand *what* it does and *why* it matters, and shows you the idiomatic .NET pattern for the situation you'll actually be in at a real job: calling out to a Python-based training or serving service from a .NET system that owns the product around it.

That last pattern — a .NET application as the product layer, calling into specialized Python services for the handful of things Python still does best — is, frankly, how most real-world .NET + LLM systems are built today. Learning to draw that boundary well is as valuable a skill as any code in this book.

The structure of the book — three levels

Exactly as in the original, people who work with LLMs split into three levels, and this book covers all three, in order:

Level 1 — Model Builder. Understanding how a neural network learns, how a transformer works, how a language model is trained from scratch — built here with your own `Value` and `Tensor` types, not someone else's library.

Level 2 — Model Customizer. Fine-tuning an existing model for a specific task, giving it new capabilities (tool use, memory), aligning its behavior — built here with TorchSharp where real training is involved, and honest conceptual coverage where it isn't yet practical in .NET.

Level 3 — Product Builder. Using all of this to build a real, reliable, evaluated, secure product — and here, C# and ASP.NET Core aren't the "adapted" language, they're the *native* one. This is the part of the stack .NET already excels at, and the part of the book where this edition can go further than merely matching the Python original.

Most tutorials start at Level 3 because it gives quick results — call an API, get a chatbot. This book starts at Level 1, because work built on a framework you don't understand never goes deep. A developer who has hand-built backpropagation in C# can look at a fine-tuning loss curve and immediately reason about what's wrong; a developer who only ever called `model.Train()` cannot.

How to read this book

Every chapter follows the same structure as the original:

1. **Plain-language concept** — the idea explained with an analogy or example, no math or code yet.
2. **Math/process** — as much as needed, no unnecessary formalism.
3. **Code** — a working, small, clean C# implementation of that concept, targeting .NET Core 10. Type and run this yourself; don't just read it.
4. **Exercise** — a task to modify or extend something yourself.
5. **Roadmap connection** — which days of the companion tracker this chapter matches.

One important piece of advice, unchanged from the original: reading the code and feeling like you understand it is not the same as having written it. Open Visual Studio or VS Code, create the project, type every line, run it, and break it on purpose. There is a large gap

between what the brain absorbs from reading and what it absorbs from typing and debugging. In Andrej Karpathy's words about the Python original this book is inspired by: *"you have to rebuild it yourself"* — that instruction doesn't change because the language did.

An honest note

There's a lot of hype around AI right now — "become an LLM expert in a weekend" claims everywhere. This book makes no such promise, and this edition doubles down on that honesty: some things genuinely don't have a clean, mature .NET story yet, and pretending they do would leave you with broken expectations the first time you tried to ship. Deeply understanding LLMs — in any language — is roughly six months of serious, consistent, hands-on work. This book, together with the companion 168-day tracker, is built to help you sustain exactly that.

What you get in return is freedom from being a permanent consumer of someone else's Python notebook. When a new paper or model architecture drops, you'll be able to read it, understand it, and — where the .NET ecosystem allows — reproduce it yourself, in the language you already ship production code in.

What you need to get started

- **.NET Core 10 SDK** (see Appendix C for install instructions and version verification)
- Visual Studio 2026 or VS Code with the C# Dev Kit extension
- NuGet access (all packages used in this book are public: `TorchSharp`, `Microsoft.ML.OnnxRuntimeGenAI`, `ModelContextProtocol`, etc. — installed chapter by chapter, only when needed)
- A GitHub account (to track and share your work)
- The companion progress tracker website (keep it open in your browser to check off daily work)
- No GPU required to start. Chapters 2–14 run comfortably on any laptop CPU. From Chapter 16 onward, a CUDA-capable GPU (or Apple Silicon via TorchSharp's Metal backend) makes runs faster but is not required to follow along conceptually.

Moving forward

In the next chapter, we start with the smallest possible building block — a number, its derivative, and a graph — expressed as a handful of lines of C#. From here, the entire world of neural networks is built, one honest layer of abstraction at a time. It won't feel complicated. You'll be surprised how much comes from how little, and that a `double` and an operator overload are all it takes to begin.

Roadmap connection: This chapter is preparation before starting Day 1. From Day 1 onward, move through it alongside Chapters 2 and 3.

ENJOYED THE SAMPLE?

Get the Full Book

This preview covered the Table of Contents and Chapter 1 only. The complete .NET C# Edition takes you all the way from a hand-rolled C# autograd engine to a production-shipped LLM system with TorchSharp, ONNX Runtime GenAI, and the official MCP C# SDK — 25 chapters, 3 appendices, ~260 pages, no Python required.

Get the .NET C# Edition →